

REST API CLIENT

SPIS TREŚCI

Spis treści	1
Cel zajęć.....	1
Rozpoczęcie	1
Uwaga	1
Wymagania.....	2
Badanie API	2
Implementacja	2
Commit projektu do GIT.....	7
Podsumowanie.....	7

CEL ZAJĘĆ

Celem głównym zajęć jest zdobycie następujących umiejętności:

- pobieranie danych z zewnętrznych zasobów za pomocą REST API
- zdobywanie wiedzy na temat zewnętrznych API za pomocą dokumentacji typu Swagger
- wysyłanie asynchronicznych żądań z wykorzystaniem XMLHttpRequest i Fetch API

W praktycznym wymiarze uczestnicy stworzą dynamiczną stronę HTML pozwalającą na wyświetlanie bieżącej informacji pogodowej oraz prognoz dla zadanej przez użytkownika miejscowości.

ROZPOCZĘCIE

Rozpoczęcie zajęć. Powtórzenie wykonywania połączeń synchronicznych i asynchronicznych z poziomu JS na stronie.

Wejściówka?

UWAGA

Ten dokument aktywnie wykorzystuje niestandardowe właściwości. Podobnie jak w LAB A wejdź do **Plik** -> **Informacje** -> **Właściwości** -> **Właściwości zaawansowane** -> **Niestandardowe** i zaktualizuj pola. Następnie uruchom ten dokument ponownie lub **Ctrl+A** -> **F9**.

WYMAGANIA

W ramach LAB D przygotowane powinny zostać:

- pojedyncza strona HTML ze skryptem ładowanym z zewnętrznego pliku JS
- pole tekstowe (input typu „text”) do wprowadzania adresu
- przycisk „Pogoda”, po kliknięciu którego wykonywane jest zapytanie asynchroniczne:
 - do API Current Weather: <https://openweathermap.org/current> za pomocą XMLHttpRequest
 - do API 5 day forecast: <https://openweathermap.org/forecast5> za pomocą Fetch API
- obsługa zwrotki z obu API – wypisanie pogody bieżącej oraz prognoz poniżej pola wyszukiwania.

Wygeneruj klucz do API. Ponieważ aktywacja może chwilę potrwać, na czas trwania laboratorium możesz wykorzystać „służbowy” klucz: `7ded80d91f2b280ec979100cc8bbba94`. **UWAGA!** Klucz zostanie dezaktywowany niedługo po zajęciach. Musisz wygenerować swój własny.

W przypadku blokady twórczej można posiłkować się filmem: <https://www.youtube.com/watch?v=WoKp2qDFxKk> jednakże spróbuj rozwiązać ten problem samodzielnie!

Prowadzący omówi powyższe wymagania. Upewnij się, czy wszystko rozumiesz.

Tu umieść swoje notatki:

...notatki...

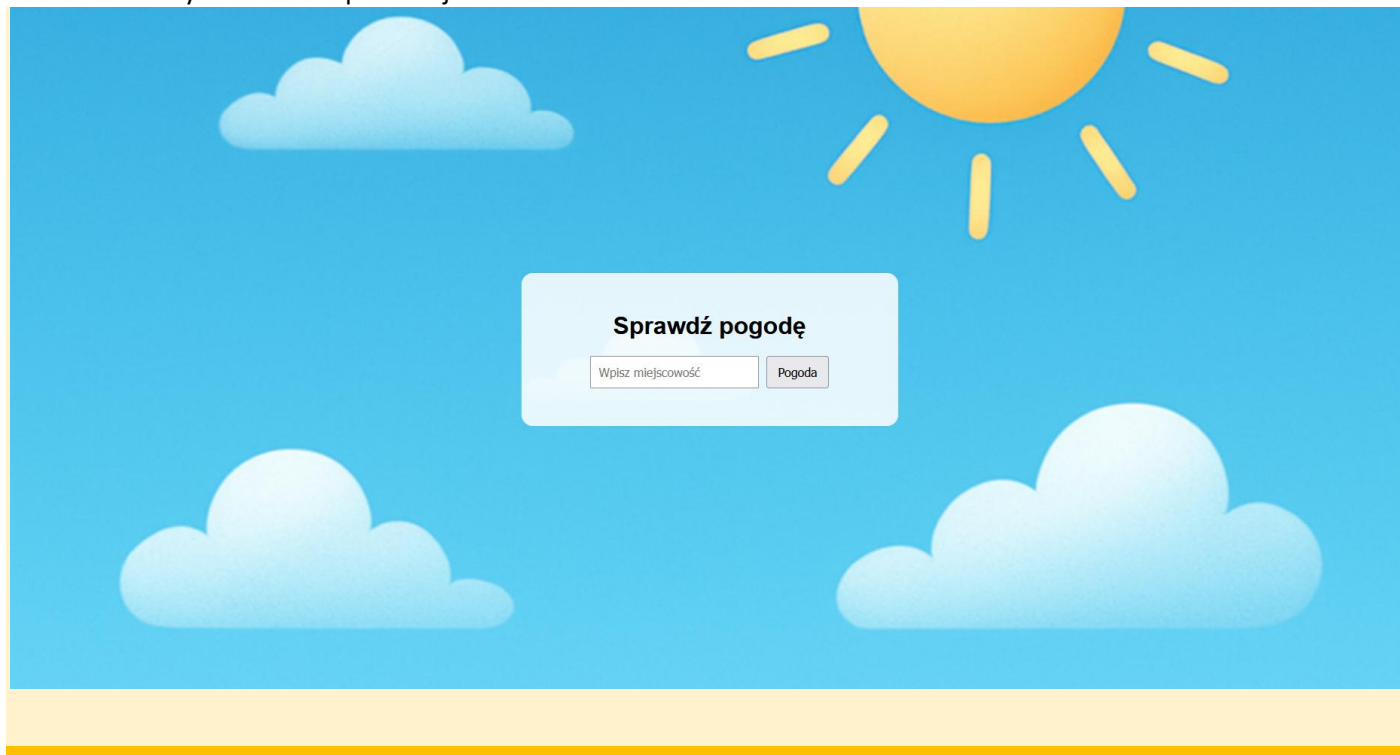
BADANIE API

Poświęć kilka minut na wykonanie przykładowych zapytań do API z poziomu pasku adresu przeglądarki. Podaj wymagane parametry dla osiągnięcia różnych wyników. Zbadaj odpowiedzi API, aby uzyskać pełen obraz wymagań i możliwości API.

IMPLEMENTACJA

Tradycyjnie implementację należy zacząć od zbudowania w HTML + CSS wszystkich wymaganych elementów / placeholderów na te elementy. Następnie krok po kroku należy implementować poszczególne zachowania.

Wstaw zrzut ekranu zawierającego stronę ze wszystkimi elementami, tj. pole tekstowe, przycisk, miejsce do wyświetlenia pogody i prognozy:

Punkty: **Poprawiam całe sprawozdanie**

0

1

Wstaw zrzut ekranu kodu odpowiedzialnego za wysyłanie żądania do current za pomocą XMLHttpRequest:

```
function getCurrentWeather(city) :void { Show usages
  const url :string = `https://api.openweathermap.org/data/2.5/weather?q=${encodeURIComponent(city)}&appid=${apiKey}&units=metric&lang=pl`;

  const xhr :XMLHttpRequest = new XMLHttpRequest();
  xhr.open( method: "GET", url, async: true);

  xhr.onload = function () :void {
    if (xhr.status === 200) {
      const data = JSON.parse(xhr.responseText);

      document.getElementById( elementId: "currentWeather").innerHTML = `
        <h2>Pogoda bieżąca</h2>
        <p><strong>${data.name}</strong></p>
        <p>Temperatura: ${data.main.temp} °C</p>
        <p>Opis: ${data.weather[0].description}</p>
      `;
    } else {
      document.getElementById( elementId: "currentWeather").innerHTML =
        "<p>Błąd pobierania pogody bieżącej</p>";
    }
  };

  xhr.onerror = function () :void {
    document.getElementById( elementId: "currentWeather").innerHTML =
      "<p>Błąd połączenia z API</p>";
  };

  xhr.send();
}
```

Wstaw zrzut ekranu pokazujący otrzymaną odpowiedź za pomocą `console.log()` w przeglądarce.

JSONNieprzetworzone daneNagłówki

ZapiszKopiujZwiń wszystkieRozwiń wszystkieFiltruj JSON

▼ coord:

lon:14.553lat:53.4289

▼ weather:

▼ 0:

id:804main:"Clouds"description:"zachmurzenie duże"icon:"04n"base:"stations"

▼ main:

temp:0.36feels_like:-3.86temp_min:-0.16temp_max:1.11pressure:1015humidity:85sea_level:1015grnd_level:1012visibility:10000

▼ wind:

speed:4.02deg:70

▼ clouds:

all:100dt:1770568442

▼ sys:

type:2id:2034200country:"PL"sunrise:1770532482sunset:1770566238timezone:3600id:3083829name:"Szczecin"cod:200

Punkty:	0	1
---------	---	---

Wstaw zrzut ekranu kodu odpowiedzialnego za wysyłanie żądania do forecast za pomocą Fetch:

```

function getForecast(city):void { Show usages
  const url:string = `https://api.openweathermap.org/data/2.5/forecast?q=${encodeURIComponent(city)}&appid=${apiKey}&units=metric&lang=pl`;

  fetch(url).then(response => {
    if (!response.ok) {
      throw new Error("Błąd HTTP: " + response.status);
    }
    return response.json();
  })
  .then(data => {
    let html:string = "<h2>Prognoza 5-dniowa</h2>";
    html += "<div class='forecast-grid'>";

    for (let i:number = 0; i < data.list.length; i += 8) {
      const day = data.list[i];
      const date:string = new Date(day.dt_txt).toLocaleDateString({ locales: "pl-PL"});

      html += `
        <div class="forecast-card">
          <strong>${date}</strong><br>
          ${day.main.temp} °C<br>
          ${day.weather[0].description}
        </div>
      `;
    }

    html += "</div>";
    document.getElementById("forecast").innerHTML = html;
  })
  .catch(error => {
    document.getElementById("forecast").innerHTML =
      "<p>Błąd pobierania prognozy</p>";
    console.error(error);
  });
}

```

Wstaw zrzut ekranu pokazujący otrzymaną odpowiedź za pomocą `console.log()` w przeglądarce.

JSON	Nieprzetworzone dane	Nagłówki
Zapisz	Kopiuj	Zwiń wszystkie Rozwiń wszystkie Filtruj JSON
cod:	"200"	
message:	0	
cnt:	40	
▼ list:		
▼ 0:		
dt:	1770573600	
▼ main:		
temp:	0.36	
feels_like:	-3.53	
temp_min:	0.32	
temp_max:	0.36	
pressure:	1015	
sea_level:	1015	
grnd_level:	1012	
humidity:	85	
temp_kf:	0.04	
▼ weather:		
▼ 0:		
id:	804	
main:	"Clouds"	
description:	"zachmurzenie duże"	
icon:	"04n"	
▼ clouds:		
all:	100	
▼ wind:		
speed:	3.57	
deg:	81	
gust:	8.8	
visibility:	10000	
pop:	0	
▼ sys:		
pod:	"n"	
dt_txt:	"2025-02-08 18:00:00"	
▼ 1:		
dt:	1770584400	
▼ main:		
temp:	0.22	
feels_like:	-4.06	
temp_min:	-0.06	
temp_max:	0.22	
pressure:	1015	
sea_level:	1015	
grnd_level:	1013	
humidity:	87	
temp_kf:	0.28	
▼ weather:		
▼ 0:		
id:	804	
main:	"Clouds"	
description:	"zachmurzenie duże"	
icon:	"04n"	
▼ clouds:		
all:	100	
▼ wind:		
speed:	4.07	
deg:	98	
gust:	9.45	
visibility:	10000	
pop:	0	
▼ sys:		
pod:	"n"	
dt_txt:	"2025-02-08 21:00:00"	
▼ 2:		
dt:	1770595200	
▼ main:		
temp:	-0.57	
feels_like:	-5.1	
temp_min:	-1.03	
temp_max:	-0.57	
pressure:	1016	
sea_level:	1016	
grnd_level:	1013	
humidity:	87	
temp_kf:	0.46	

Punkty:	0	1
---------	---	---

Wstaw zrzut ekranu przedstawiającego wizualizację prognoz pogody:



Upewnij się, że widoczne są pasek wyszukiwania ze wskazaną miejscowością, a także zarówno pogoda bieżąca jak i prognozy pogody.

Punkty:	0	1
---------	---	---

COMMIT PROJEKTU DO GIT

Zacommituj i pushnij swoje rozwiązanie do repozytorium GIT.

Upewnij się, czy wszystko dobrze się wysłało. Jeśli tak, to z poziomu przeglądarki utwórz branch o nazwie `lab-d` na podstawie głównej gałęzi kodu.

Podaj link do brancha `lab-d` w swoim repozytorium:
<https://github.com/Pipulski2001/APPINT/tree/lab-d>

PODSUMOWANIE

W kilku słowach/zdaniach napisz swoje przemyślenia odnośnie tego laboratorium. Nie używaj LLM.

Zweryfikuj kompletność sprawozdania. Utwórz PDF i wyślij w terminie.